

Explicacion de la Implementacion de uart_tx

1. Objetivo del modulo

El archivo `src/uart_tx.v` implementa un transmisor UART sencillo. Su trabajo es tomar un byte de entrada (`tx_data`) y enviarlo por la linea serie `tx` usando el formato clasico:

- 1 bit de inicio (`start bit`) con valor 0
- 8 bits de datos
- 1 bit de parada (`stop bit`) con valor 1

Mientras se transmite el byte, la salida `tx_busy` indica que el transmisor esta ocupado.

2. Interfaz del modulo

```
module uart_tx #(
    parameter CLK_FREQ = 27_000_000,
    parameter BAUD_RATE = 115200
)(
    input wire clk,
    input wire rst_n,
    input wire tx_en,
    input wire [7:0] tx_data,
    output reg tx,
    output reg tx_busy
);
```

Entradas

- `clk`: reloj principal del sistema.
- `rst_n`: reset activo en bajo. Cuando vale 0, el modulo vuelve a su estado inicial.
- `tx_en`: pulso o habilitacion que ordena iniciar una transmision.
- `tx_data`: byte que se desea transmitir.

Salidas

- `tx`: linea serie de salida.
- `tx_busy`: indica si el modulo esta enviando un dato.

3. Parametros

```
localparam CLOCKS_PER_BIT = CLK_FREQ / BAUD_RATE;
```

Esta constante define cuantos ciclos de reloj debe durar cada bit en la UART. Por ejemplo, si:

$$\text{CLK_FREQ} = 27\,000\,000 \quad \text{y} \quad \text{BAUD_RATE} = 115200$$

entonces:

$$\text{CLOCKS_PER_BIT} \approx \frac{27\,000\,000}{115200} = 234$$

Eso significa que cada bit se mantiene estable durante 234 ciclos de `clk`.

4. Estados de la maquina

```
localparam s_IDLE = 2'b00;  
localparam s_START = 2'b01;  
localparam s_DATA = 2'b10;  
localparam s_STOP = 2'b11;
```

El modulo esta organizado como una maquina de estados finitos con cuatro etapas:

- `s_IDLE`: espera un nuevo dato para transmitir.
- `s_START`: envia el bit de inicio.
- `s_DATA`: envia los 8 bits del byte.
- `s_STOP`: envia el bit de parada.

5. Registros internos

```
reg [1:0] state;  
reg [15:0] clk_count;  
reg [2:0] bit_index;  
reg [7:0] tx_data_reg;
```

Cada registro tiene una funcion especifica:

- `state`: guarda el estado actual de la maquina.

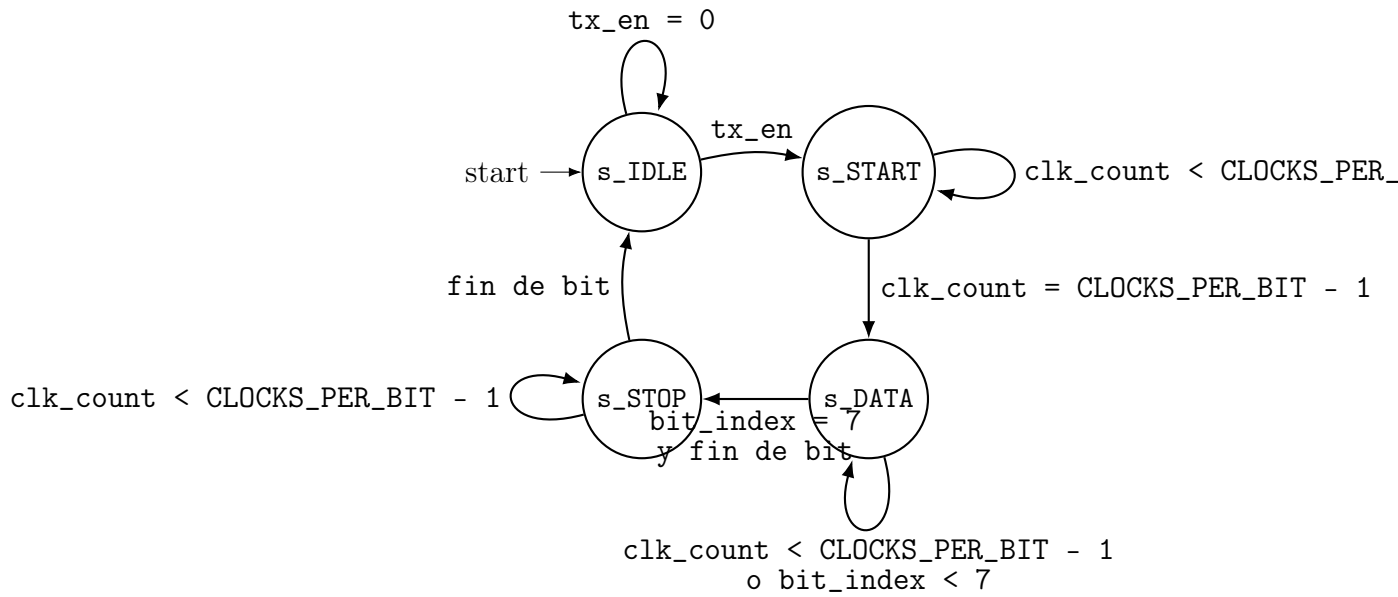


Figura 1: Diagrama visual de la FSM del transmisor UART.

- `clk_count`: cuenta cuantos ciclos de reloj han pasado dentro del bit actual.
- `bit_index`: indica que bit del byte se esta transmitiendo, desde 0 hasta 7.
- `tx_data_reg`: almacena una copia local del byte a transmitir.

Guardar el dato en `tx_data_reg` es importante porque la entrada `tx_data` podria cambiar mientras la transmision esta en curso.

6. Bloque secuencial principal

```
always @(posedge clk or negedge rst_n)
```

Todo el comportamiento del transmisor ocurre en este bloque secuencial. Eso significa que:

- el modulo actualiza sus registros en cada flanco positivo del reloj,
- y si `rst_n` baja a 0, se reinicia de inmediato.

7. Comportamiento durante reset

```
if (!rst_n) begin
    state <= s_IDLE;
    clk_count <= 0;
    bit_index <= 0;
    tx_data_reg <= 8'h00;
    tx <= 1'b1;
    tx_busy <= 1'b0;
end
```

Cuando ocurre reset:

- el estado vuelve a `s_IDLE`,
- la cuenta de reloj se limpia,
- el índice del bit vuelve a cero,
- la copia local del dato se borra,
- la línea `tx` queda en 1,
- y `tx_busy` queda en 0.

Que `tx` quede en 1 es correcto, porque en UART la línea en reposo se mantiene alta.

8. Estado `s_IDLE`

```
s_IDLE: begin
    tx <= 1'b1;
    clk_count <= 0;
    bit_index <= 0;

    if (tx_en) begin
        tx_data_reg <= tx_data;
        tx_busy <= 1'b1;
        state <= s_START;
    end else begin
        tx_busy <= 1'b0;
        state <= s_IDLE;
    end
end
```

Este es el estado de espera.

- La salida `tx` se mantiene en 1, porque no se está transmitiendo nada.
- `clk_count` y `bit_index` se reinician para preparar una transmisión nueva.
- Si `tx_en` se activa, el módulo captura `tx_data`, marca `tx_busy = 1` y pasa a `s_START`.
- Si `tx_en` no está activo, el módulo sigue esperando.

En otras palabras, `s_IDLE` detecta el inicio de una nueva operación.

9. Estado s_START

```
s_START: begin
    tx <= 1'b0;
    if (clk_count < CLOCKS_PER_BIT - 1) begin
        clk_count <= clk_count + 1;
        state <= s_START;
    end else begin
        clk_count <= 0;
        state <= s_DATA;
    end
end
```

Aqui se envia el bit de inicio.

- La salida tx se fuerza a 0.
- clk_count se incrementa ciclo a ciclo.
- Mientras no se cumpla la duracion completa del bit, el estado no cambia.
- Al terminar el conteo, se reinicia clk_count y se pasa a s_DATA.

La idea principal es que el bit de inicio dure exactamente un tiempo de bit.

10. Estado s_DATA

```
s_DATA: begin
    tx <= tx_data_reg[bit_index];
    if (clk_count < CLOCKS_PER_BIT - 1) begin
        clk_count <= clk_count + 1;
        state <= s_DATA;
    end else begin
        clk_count <= 0;
        if (bit_index < 7) begin
            bit_index <= bit_index + 1;
            state <= s_DATA;
        end else begin
            bit_index <= 0;
            state <= s_STOP;
        end
    end
end
```

Este es el bloque mas importante, porque aqui se transmiten los 8 bits del byte.

Que se envia

La linea:

```
tx <= tx_data_reg[bit_index];
```

hace que en la salida aparezca el bit seleccionado por `bit_index`. Como `bit_index` empieza en 0, el modulo envia primero el bit menos significativo (LSB). Ese comportamiento coincide con el protocolo UART clasico.

Como avanza

- Durante un tiempo de bit, `clk_count` cuenta los ciclos del reloj.
- Cuando termina ese tiempo, el modulo decide si debe pasar al siguiente bit o terminar la carga util.
- Si `bit_index < 7`, incrementa el indice y permanece en `s_DATA`.
- Si ya se transmitio el bit 7, reinicia `bit_index` y cambia a `s_STOP`.

11. Estado `s_STOP`

```
s_STOP: begin
    tx <= 1'b1;
    if (clk_count < CLOCKS_PER_BIT - 1) begin
        clk_count <= clk_count + 1;
        state <= s_STOP;
    end else begin
        clk_count <= 0;
        state <= s_IDLE;
    end
end
```

En este estado se envia el bit de parada:

- la linea `tx` vuelve a 1,
- se espera un tiempo de bit completo,
- y despues el modulo regresa a `s_IDLE`.

Cuando vuelve a `s_IDLE`, el transmisor ya esta listo para aceptar otro byte.

12. Secuencia completa de transmision

Si se quiere transmitir, por ejemplo, el byte:

$$\text{tx_data} = 8'b1010\ 1100$$

entonces el orden en la linea `tx` sera:

1. reposo: 1
2. bit de inicio: 0
3. bit 0

4. bit 1
5. bit 2
6. bit 3
7. bit 4
8. bit 5
9. bit 6
10. bit 7
11. bit de parada: 1

Como UART transmite primero el bit menos significativo, el orden real de los bits de datos sera:

0, 0, 1, 1, 0, 1, 0, 1

13. Papel de tx_busy

La senal `tx_busy` se activa cuando el modulo detecta `tx_en` en `s_IDLE`. Su objetivo es informar que el transmisor esta ocupado y no deberia recibir una nueva peticion.

En este diseno, `tx_busy` se limpia al volver a `s_IDLE`. Eso hace que sea una bandera sencilla de control para el resto del sistema.

14. Resumen funcional

El funcionamiento general puede resumirse asi:

1. el modulo espera en `s_IDLE`,
2. cuando `tx_en` se activa, guarda el byte,
3. envia un bit de inicio en `s_START`,
4. envia los 8 bits del dato en `s_DATA`,
5. envia un bit de parada en `s_STOP`,
6. y vuelve a reposo.

15. Observaciones importantes para entender el código

- El diseño usa una sola máquina de estados, por eso es fácil seguir su flujo.
- La temporización de la UART depende directamente de `CLOCKS_PER_BIT`.
- El envío de datos es `LSB first`, como es habitual en UART.
- No hay bit de paridad ni múltiples bits de parada; es una implementación básica de tipo 8N1.
- El módulo asume que `tx_en` se activa cuando el transmisor está libre.

16. Conclusion

La implementación de `uart_tx.v` es un transmisor UART compacto y claro. Internamente combina:

- una máquina de estados para organizar la secuencia de envío,
- un contador para fijar la duración de cada bit,
- un índice para recorrer los 8 bits del byte,
- y un registro auxiliar para conservar el dato durante toda la transmisión.

Desde el punto de vista de aprendizaje, este módulo es una buena base para entender comunicación serial, temporización por baud rate y diseño secuencial en Verilog.